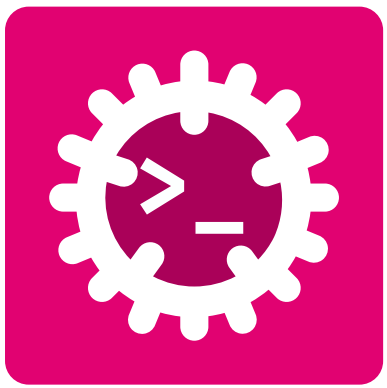
Hes·so VALAIS WALLIS

School of Engineering

SYSTEMS ENGINEERING INFOTRONICS



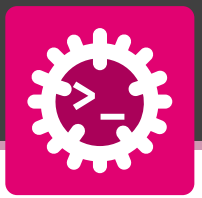
Advanced Programming

Object-Oriented Programming

Silvan Zahno

University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

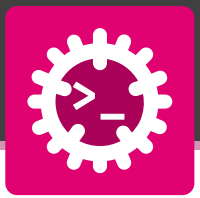
2026-10-07 - v0.1.0



Contents

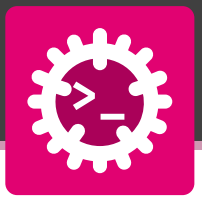
Object-Oriented Programming 4
What is an Object? 5
Class vs Object 6
The Four Pillars of Object-Oriented Programming (OOP) 7
The Four Pillars of OOP in Rust 8
Exercise - Objects 9
Building a Class in Rust 10
A Class = `struct` + `impl` 11
Objects = Instances 12
Methods & the `self` Receiver 13
Constructors 14
The `Default` Trait 15
Exercise - Building a Class 16
Pillar 1 - Encapsulation 17
Encapsulation: Private Data, Public API 18

Contents



Visibility & Modules 19
Getters & Setters 20
Exercise - Encapsulation 21
Pillar 2 - Abstraction 22
Traits: the Contract 23
Programming to the Abstraction 24
`impl Trait` in Argument Position 25
Multiple Traits & Supertraits 26
Exercise - Abstraction 27
Pillar 3 - Inheritance 28
Rust Has No Class Inheritance 29
Composition over Inheritance 30
Exercise - Composition 31
Pillar 4 - Polymorphism 32
Polymorphism: One Name, Many Types 33
Static Dispatch: Generics & Trait Bounds 36

Contents



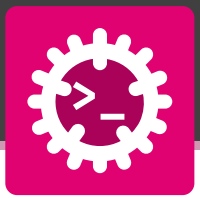
Dynamic Dispatch: Trait Objects 37
Static vs Dynamic Dispatch 38
Exercise - Polymorphism 39
OOP in Rust - Recap 40
Glossary & Bibliography 43



Object-Oriented Programming

Teaching data to behave

What is an Object?



An **object** bundles **state** (data) with the **behaviour** (operations) that acts on that data. A program becomes a set of objects collaborating through their public operations.

Procedural view - data and the functions on it live apart:

```
struct Account {
  balance: u64
}

// free functions, anyone can touch the field
fn deposit(a: &mut Account, n: u64) {
  a.balance += n;
}
fn balance(a: &Account) -> u64 {
  a.balance
}
```

Object view - data and behaviour belong together:

```
struct Account {
  balance: u64
}

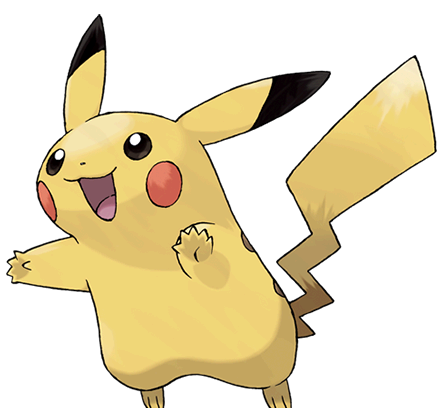
impl Account {                    // behaviour attached
  fn deposit(&mut self, n: u64) {
    self.balance += n;
  }
  fn balance(&self) -> u64 {
    self.balance
  }
}
```

- Same data, but the operations now **belong to the type**
- This bundling is the seed of every pillar that follows

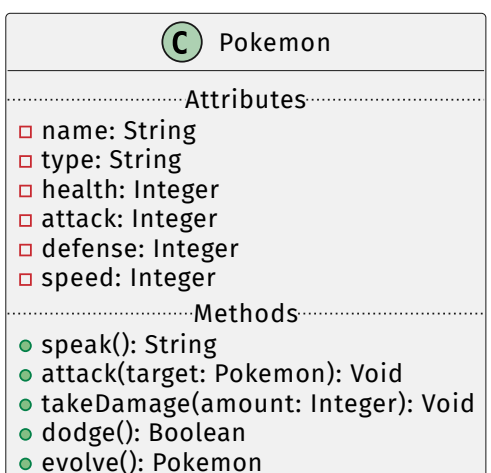
Class vs Object

A **class** is the blueprint - the **type** and its behaviour. An **object** is one concrete **instance** built from that blueprint.

Object



Class

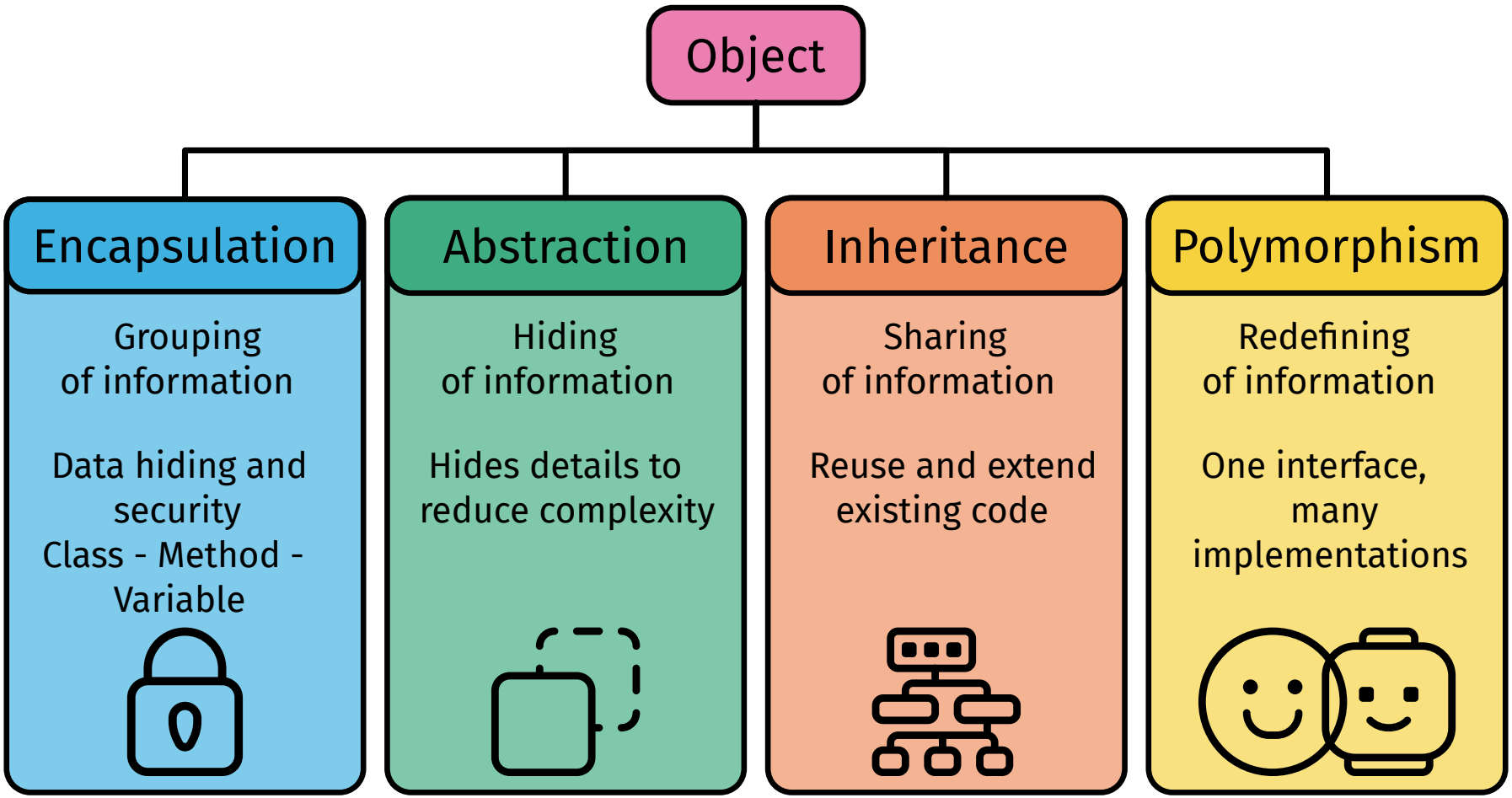


Attributes

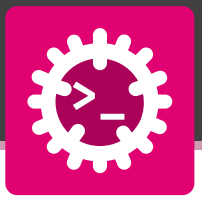
Methods

- **Class** - written once: `Pokemon` defines what every Pokémon **is** and **can do**
- **Object** - created many times: `pikachu`, `charmander`, … each with its own field values
- All objects of a class **share the same behaviour** but hold **independent state**

The Four Pillars of OOP



The Four Pillars of OOP in Rust



Classic OOP rests on four pillars.

C++ supports all four directly through the class model:

- **Encapsulation** ⇨ `class` + `private` / `public` access specifiers
- **Abstraction** ⇨ abstract classes with pure `virtual` functions
- **Inheritance** ⇨ public inheritance (`class Dog : public Animal`)
- **Polymorphism** ⇨ `templates` (compile-time) + `virtual` functions (run-time)



Rust keeps the **goals** but changes the **tools**:

- **Encapsulation** ⇨ modules + `pub` visibility
- **Abstraction** ⇨ `trait` (the contract)
- **Inheritance** ⇨ *politely declined* - replaced by `traits` + composition
- **Polymorphism** ⇨ `generics` (static dispatch) + `trait objects` (dynamic dispatch)



> We meet each pillar in turn, then recap the full mapping at the end.

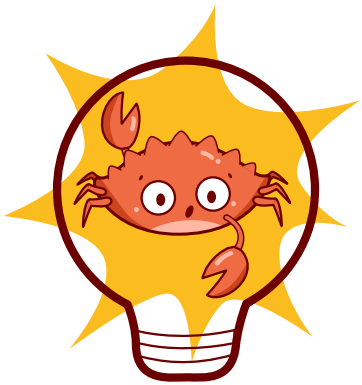
Exercise - Objects





A smart **Lamp** can be switched on or off and dimmed from `0` to `100` %.

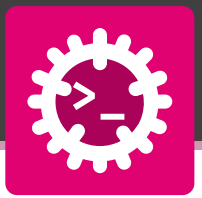
1. List its **state** - the data the object holds.
2. List its **behaviour** - the operations acting on that data.
3. Which part becomes the `struct`, which becomes the `impl`?



Building a Class in Rust

`struct` for data / `impl` for behaviour

A Class = `struct` + `impl`

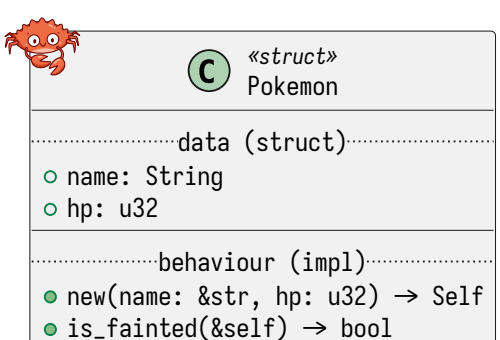


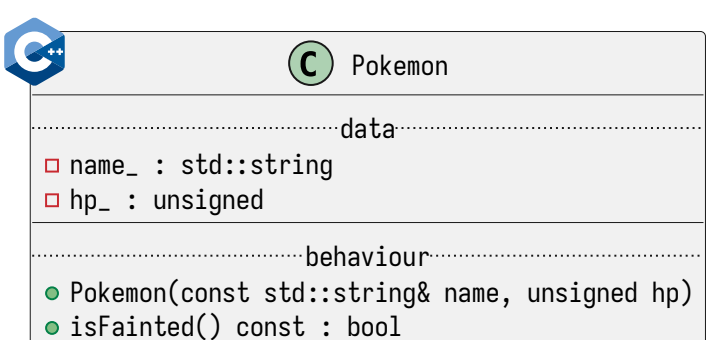
A class splits into two halves: a **`struct`** holds the **data**, an **`impl`** block holds the **behaviour**.

```rust
struct Pokemon { name: String, hp: u32 }

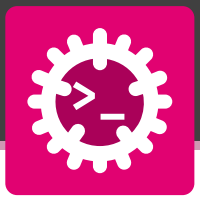
// inherent behaviour can be split across blocks
impl Pokemon {
    fn new(name: &str, hp: u32) -> Self {        // construction
        Self { name: name.into(), hp }
    }
}
impl Pokemon {
    fn is_fainted(&self) -> bool { self.hp == 0 } // queries
}
```

- **Field / attribute** ⇨ a member of the **`struct`**
- **Method** ⇨ a function in an **`impl`** block whose first parameter is **`self`**
- A type may have **several** **`impl`** blocks - group behaviour as you like
- Data and behaviour are written **separately** but belong to the same type





Objects = Instances



An object is a **value** of the type. Create one, then call its methods.

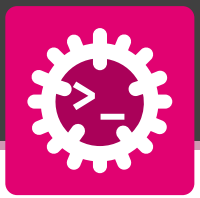
```
fn main() {
  // two independent objects of the same class
  let pikachu    = Pokemon {
    name: "Pikachu".into(), hp: 35
  };
  let charmander = Pokemon::new("Charmander", 39);

  println!("{}", pikachu.name);            // own state
  println!("{}", charmander.is_fainted()); // shared behaviour
}
```

- One **class**, many **objects** - each carries its **own** field values
- All objects share the **same** methods

```
// The same as calling the method on the object
pikachu.is_fainted()
// Is equivalent to the fully qualified call
Pokemon::is_fainted(&pikachu)
```

Methods & the `self` Receiver



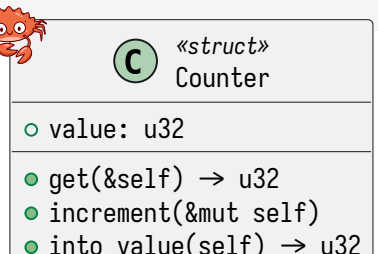
The first parameter - the **receiver** - decides whether a method **reads**, **modifies**, or **consumes** the object.

```rust
struct Counter { value: u32 }

impl Counter {
  fn get(&self) -> u32 { self.value }          // read
  fn increment(&mut self) { self.value += 1; } // modify
  fn into_value(self) -> u32 { self.value }    // consume
}

fn main() {
  let mut c = Counter { value: 0 };
  c.increment();           // borrows &mut c
  println!("{}", c.get()); // borrows &c  arrow.stroked1
  let v = c.into_value();  // moves c
  // c.get();              // × c was consumed
  println!("{v}");         // 1
}
```

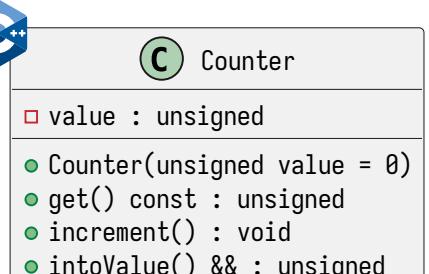




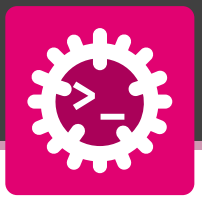
`&self`	borrow to **read**
`&mut self`	borrow to **modify**
`self`	take ownership (**consume**)
none	associated fn ⇨ `Type::f()`

- `Self` (capital) - alias for the type
- Prefer `&self`; use `self` only when the value must be consumed





Constructors



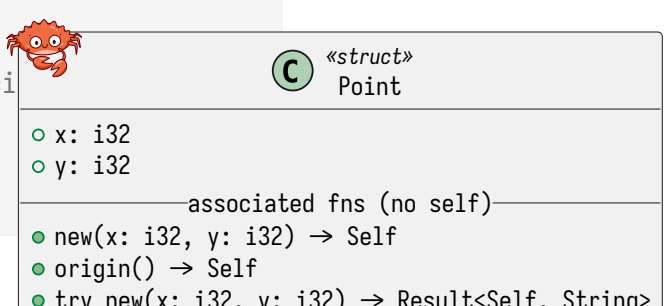
A **constructor** builds a value and puts it in a **valid state**. Rust has **no constructor keyword** - by convention an **associated function** (no `self`) named `new` returns `Self`.

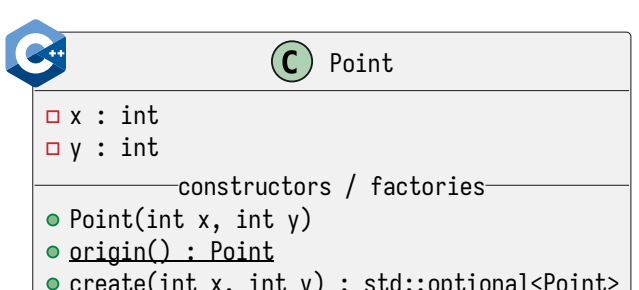
```rust
struct Point { x: i32, y: i32 }

impl Point {
  fn new(x: i32, y: i32) -> Self { Self { x, y } }
  fn origin() -> Self { Self { x: 0, y: 0 } } // named constructor
  // fallible construction returns a Result
  fn try_new(x: i32, y: i32) -> Result<Self, String> {
    if x ≥ 0 && y ≥ 0 { Ok(Self { x, y }) }
    else { Err("coords must be ≥ 0".into()) }
  }
}

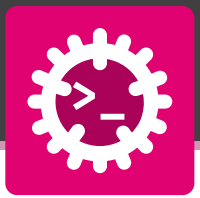
fn main() {
  let a = Point::new(1, 2); // Type::fn() ⇒ associ
  let b = Point::origin();
  let c = Point::try_new(-1, 0); // Err(...)
}
```

- **Constructor** ⇨ associated fn returning `Self`, called `Type::new()`
- Offer **several** named constructors (`origin`, `from_tuple`, …)
- **Fallible** construction ⇨ return `Result<Self, E>` instead of throwing
- No hidden default constructor - you write the ones you want





The `Default` Trait



For a **no-argument** constructor, Rust offers a standard `Default` trait - usually just **derived**.

```
#[derive(Default, Debug)]
struct Config {
  verbose: bool,   // → false
  retries: u32,    // → 0
}
// Custom default when 0 and false aren't right
struct ServerConfig { port: u16, host: String }
impl Default for ServerConfig {
  fn default() -> Self {
    Self { port: 8080, host: "localhost".into() }
} }
fn main() {
  let c = Config::default();
  println!("{c:?}");  // verbose: false, retries: 0

  // struct-update: set a few fields, default the rest
  let c2 = Config { retries: 3, ..Default::default() };
  println!("{c2:?}"); // verbose: false, retries: 3

  let s = ServerConfig::default();
  println!("{}:{}", s.host, s.port); // localhost:8080
}
```

- `#[derive(Default)]` works when **every field** implements `Default`:
  - numbers ⇨ `0`
  - `bool` ⇨ `false`
  - `String`/`Vec` ⇨ empty
- Call it for free as `Type::default()`
- **Struct update** `..Default::default()` fills the untouched fields
- Need a non-zero default? Implement `Default` by hand - a **trait impl**, which we meet under Abstraction

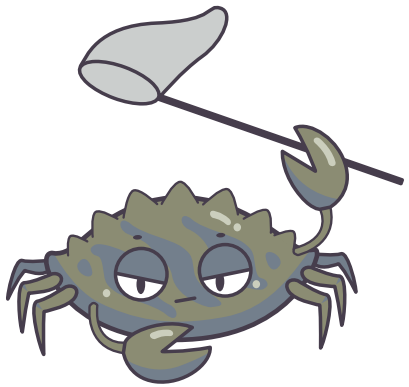
Exercise - Building a Class





Build the `Lamp` class:

1. A `struct Lamp` with fields `on: bool` and `brightness: u32`.
2. A constructor `new()` returning a lamp that is **off** at brightness `0`.
3. Methods `turn_on(&mut self)` (sets `on = true`) and `is_on(&self) -> bool`.
4. In `main`, create a lamp, turn it on and print `is_on()`.



Pillar 1 - Encapsulation

Hide the wiring, expose the buttons

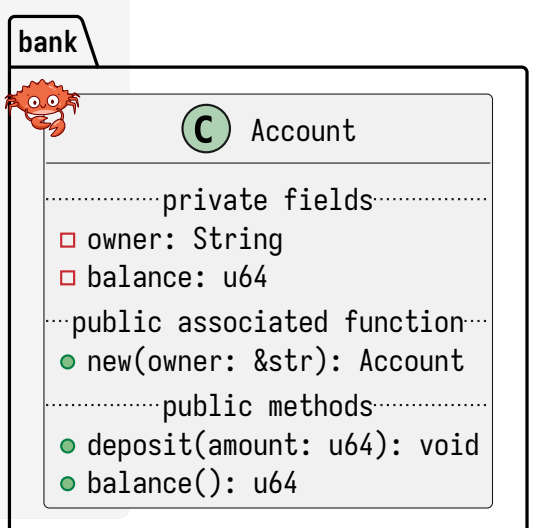
Encapsulation: Private Data, Public API

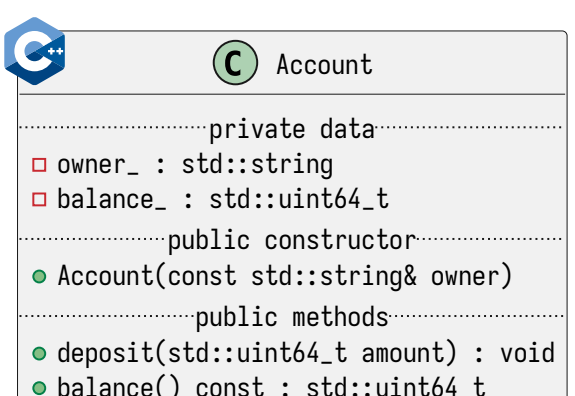
Keep fields **private**; let callers change state only through a small **public** set of methods.

```rust
mod bank {
  pub struct Account {
    owner: String,   // private - hidden outside `bank`
    balance: u64,
  }
  impl Account {
    pub fn new(owner: &str) -> Self {
      Self { owner: owner.into(), balance: 0 }
    }
    pub fn deposit(&mut self, amount: u64) {
      self.balance += amount;          // mutate via method
    }
    pub fn balance(&self) -> u64 { self.balance }
  }
}

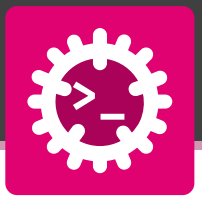
fn main() {
  let mut acc = bank::Account::new("Ada");
  acc.deposit(100);
  println!("{}", acc.balance()); // 100
  // acc.balance += 1;  // × field is private
}
```

- **Encapsulation** ⇨ bundle data + methods, expose only a public API
- **Visibility** ⇨ pub opt-in, **private by default**
- **Data protection** ⇨ outside code cannot corrupt **balance** directly





Visibility & Modules

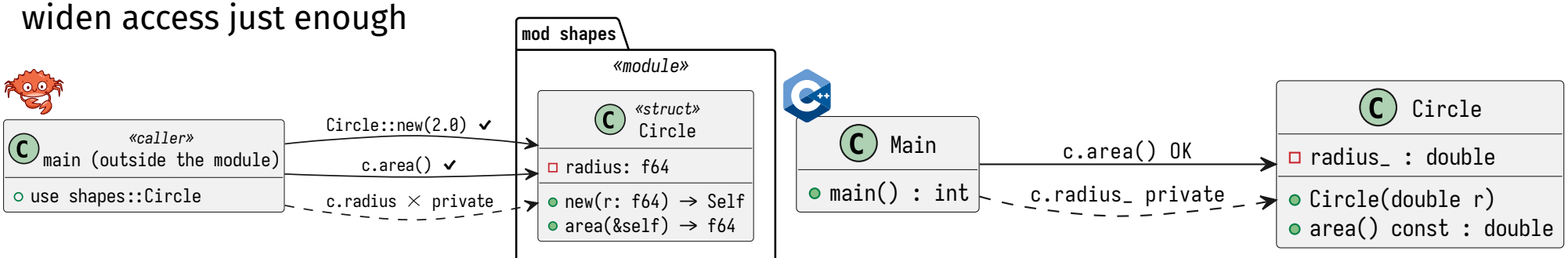


Privacy in Rust is enforced by the **module**, not by the class.

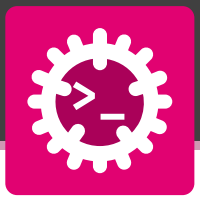
- **Private by default** - an item is visible only inside its own **module**
- **pub** opens it up - applies to a `struct`, an `fn`, a single **field**, or a `trait`
- A `pub struct` with **private fields** exposes the **type** but hides its **data** - callers must go through methods
- Hiding internals lets you **change them later** without breaking callers
- Finer control exists: `pub(crate)`, `pub(super)` to widen access just enough

```rust
mod shapes {
  pub struct Circle {      // type is public
    radius: f64,           // field stays private
  }
  impl Circle {
    pub fn new(r: f64) -> Self { Self { radius: r } }
    pub fn area(&self) -> f64 { 3.14159 * self.radius * self.radius }
  }
}  // end of the module shapes

use shapes::Circle;
let c = Circle::new(2.0);  // OK
// let r = c.radius;       // × private field
```



Getters & Setters



Private fields plus a **getter** (read) and a **setter** (write) let the type **enforce its invariants**.

```rust
pub struct Temperature { celsius: f64 } // private field

impl Temperature {
  pub fn new(celsius: f64) -> Self { Self { celsius } }

  // getter - Rust idiom: name it after the field
  pub fn celsius(&self) -> f64 { self.celsius }

  // setter - validate before mutating
  pub fn set_celsius(&mut self, value: f64) {
    if value ≥ -273.15 {        // physical lower bound
      self.celsius = value;
    }
  }
}

fn main() {
  let mut t = Temperature::new(20.0);
  t.set_celsius(25.0);   println!("{}", t.celsius()); // 25
  t.set_celsius(-300.0); // rejected, stays 25
}
```

- **Getter** ⇨ read access, named after the field (**no** `get_` prefix in Rust)
- **Setter** ⇨ `set_xxx(&mut self, ...)`, the place to **validate**
- Protects the **invariants** the struct must always uphold
- Return `&T` (not a clone) when the field is large



Don't add getters/setters by reflex - expose only what the API truly needs.

Exercise - Encapsulation





Make the brightness **impossible** to set out of range `0..=100`.

1. Keep the field `brightness` **private**.
2. Add a getter `brightness(&self) -> u32`.
3. Add a setter `set_brightness(&mut self, pct: u32)` that **clamps** anything above `100` to `100`.
4. Show that writing `lamp.brightness = 250;` from outside would **not compile**.



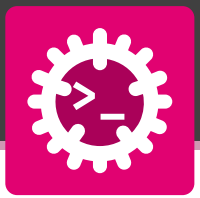
A private field plus a validating setter is how a type **protects its invariants**.



Pillar 2 - Abstraction

Program to ideas, not details

Traits: the Contract



A **trait** states **what** a type can do; each type decides **how**. It is Rust's **interface**.

```rust
// the abstraction: a contract, no data
trait Shape {
  fn area(&self) -> f64;              // required (abstract method)
  fn name(&self) -> &str { "shape" } // default method
}

struct Circle { r: f64 }
struct Rect   { w: f64, h: f64 }

impl Shape for Circle {
  fn area(&self) -> f64 { 3.14159 * self.r * self.r }
  fn name(&self) -> &str { "circle" }   // override default
}
impl Shape for Rect {
  fn area(&self) -> f64 { self.w * self.h } // keep default name
}
```

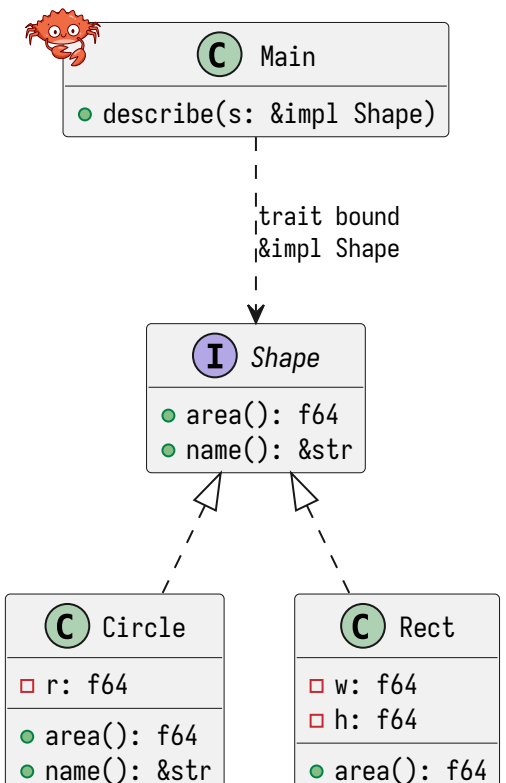
- **Interface / contract** ⇨ the trait's **required** methods
- **Abstract method** ⇨ a required method, no body
- **Default method** ⇨ a method **with** a body; types may override it
- **Implementation** ⇨ `impl Trait for Type`
- Each type supplies its **own** `area()` - the **how** is hidden behind the **what**

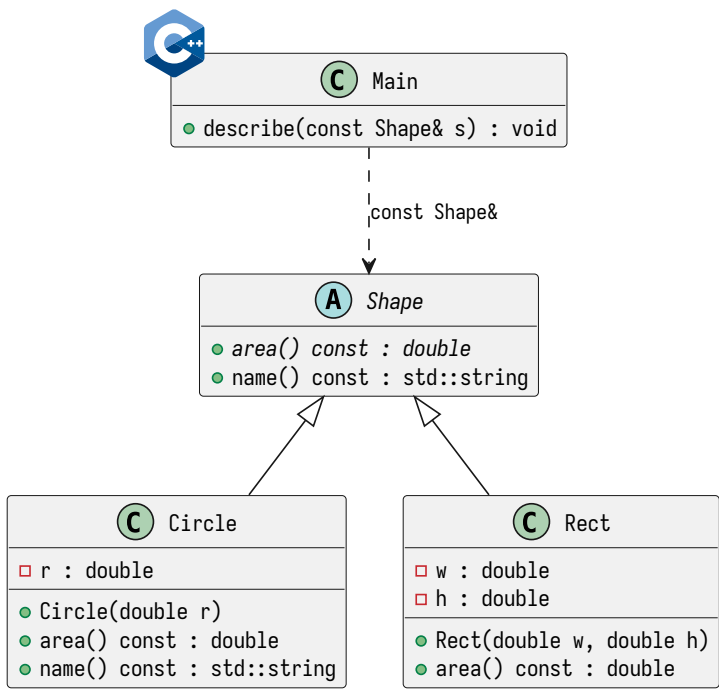
Programming to the Abstraction

Write functions against the **trait**, not the concrete type - any present or future implementor fits.

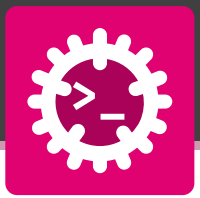
```rust
// the caller knows only the contract `Shape`
fn describe(s: &impl Shape) {
  println!("{} area = {:.2}", s.name(), s.area());
}
fn main() {
  let c = Circle { r: 2.0 };
  let r = Rect   { w: 3.0, h: 4.0 };
  describe(&c);  // circle area = 12.57
  describe(&r);  // shape  area = 12.00
}
```

The caller depends on the **trait**, not the types ⇨ **open for extension**: add a new `Shape` and `describe` just works.





`impl Trait` in Argument Position



`&impl Shape` is **shorthand** for a generic with a trait bound - same machinery, shorter spelling.

Shorthand - **`impl Trait`** in the argument:

```
fn describe(s: &impl Shape) {
  println!("{} = {:.2}", s.name(), s.area());
}
```

Desugared - an explicit generic + trait bound:

```
fn describe<T: Shape>(s: &T) {
  println!("{} = {:.2}", s.name(), s.area());
}
```

The compiler treats these as **identical**.

- **`&impl Shape`** reads as **"a reference to some type that implements Shape"**
- The concrete type is chosen **per call site**, resolved at **compile time** (static dispatch)
- **Trade-off:** **`impl Trait`** can't be **named**, so it can't tie two arguments to the **same** type:

```
// two DIFFERENT Shapes allowed:
fn pair(a: impl Shape, b: impl Shape)
// forces a and b to the SAME type:
fn pair<T: Shape>(a: T, b: T)
```

- Contrast with **`&dyn Shape`** - a **trait object**, resolved at **runtime** (covered under Polymorphism)

Multiple Traits & Supertraits

A type can keep **many** contracts at once, and traits can **build on** other traits.

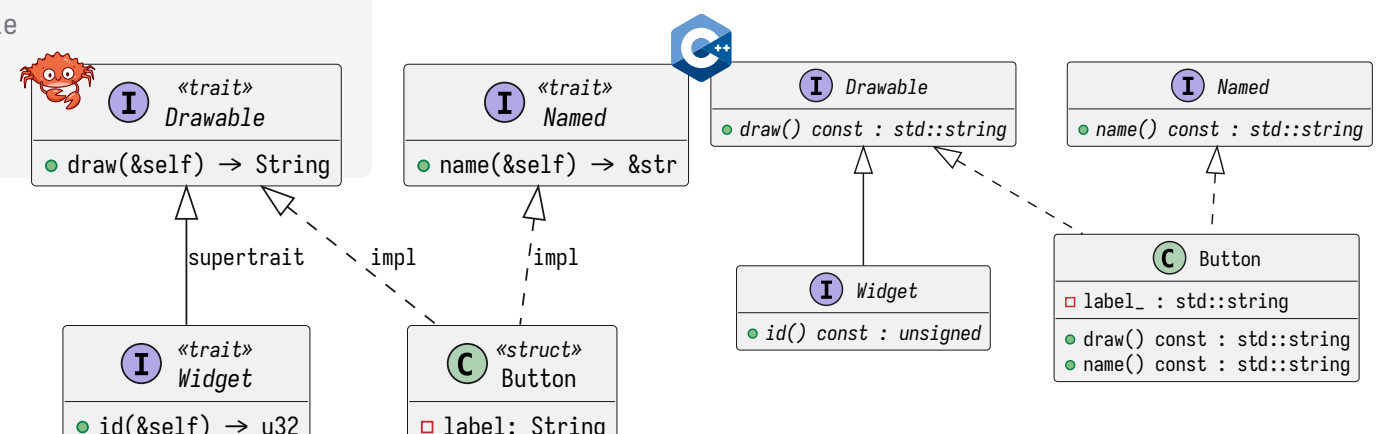
```rust
trait Drawable { fn draw(&self) -> String; }
trait Named    { fn name(&self) -> &str; }

struct Button { label: String }

// one type implements several interfaces
impl Drawable for Button {
  fn draw(&self) -> String { format!("Button({})", self.label) }
}
impl Named for Button {
  fn name(&self) -> &str { &self.label }
}

// supertrait: a Widget must ALSO be Drawable
trait Widget: Drawable {
  fn id(&self) -> u32;
}
```

- **Multiple interfaces** ⇨ implement many traits per type (cf. multiple inheritance - but of **behaviour**, never of **data**)
- **Supertrait** ⇨ `trait Widget: Drawable` requires `Drawable` too
- **Dependency inversion** ⇨ depend on trait bounds, not concrete types



Exercise - Abstraction



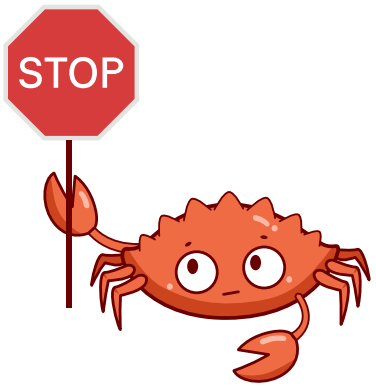


Give different devices **one shared contract** with a trait.

1. Define `trait Device { fn status(&self) -> String; }`.
2. Implement it for `Lamp` (`"lamp on"` / `"lamp off"`) and for `Thermostat` (`"22 °C"`).
3. Write `report(d: &impl Device)` that prints the status.
4. Call `report` with one lamp and one thermostat.



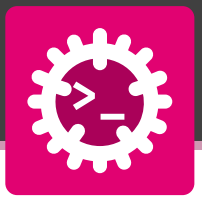
Writing `report` against the **trait** means any future `Device` works without changing it.



Pillar 3 - Inheritance

The pillar Rust politely declines

Rust Has No Class Inheritance



There is **no** `struct Dog : Animal`. Shared **behaviour** comes from **traits with default methods**; shared **data** comes from **composition**.

```rust
// ✗ not valid Rust:
// struct Dog : Animal { ... }

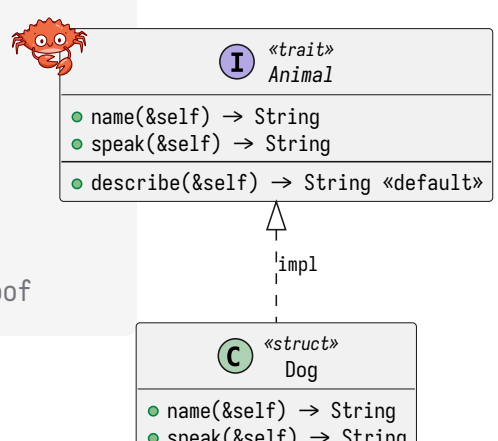
// ✓ a trait + a default method = shared behaviour
trait Animal {
  fn name(&self)  -> String;
  fn speak(&self) -> String;
  fn describe(&self) -> String {        // "inherited" behaviour
    format!("{} says {}", self.name(), self.speak())
  }
}

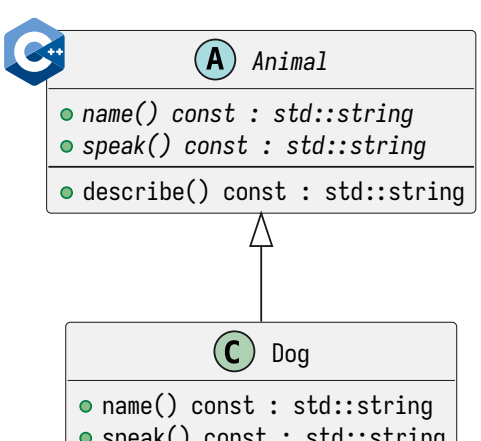
struct Dog;
impl Animal for Dog {
  fn name(&self)  -> String { "Dog".into() }
  fn speak(&self) -> String { "woof".into() }
}

fn main() { println!("{}", Dog.describe()); } // Dog says woof
```

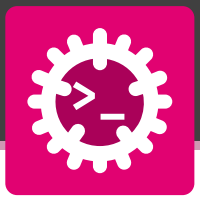
Mapping the inheritance vocabulary:

- **Base class** ⇨ a `trait`
- **Derived class** ⇨ a type that `impl`s it
- **Override** ⇨ the type's own method body
- **Extend** ⇨ implement **more** traits
- **Reuse** ⇨ default methods - **behaviour only**, never inherited fields





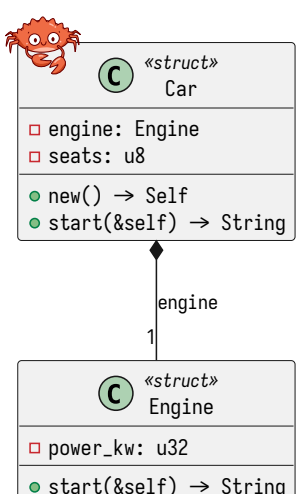
Composition over Inheritance

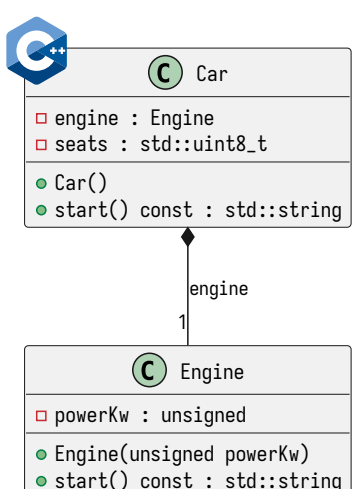


Instead of inheriting, a type contains other types (**has-a**) as fields and **delegates** work to them - reusing their **data and behaviour** without subclassing. e.g. Car **has-a** Engine.

```rust
struct Engine { power_kw: u32 }
impl Engine {
  fn start(&self) -> String {
    format!("{} kW engine running", self.power_kw)
} }
struct Car {
  engine: Engine,   // has-a Engine
  seats: u8,
}
impl Car {
  fn new() -> Self {
    Self { engine: Engine { power_kw: 150 }, seats: 5 }
  }
  // delegation: forward to the inner part
  fn start(&self) -> String { self.engine.start() }
}
fn main() {
  println!("{}", Car::new().start());
}                              // 150 kW engine running
```

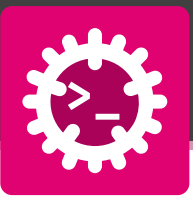
- **Has-a relationship** ⇨ fields hold other types
- **Delegation** ⇨ forward calls to an inner field
- **Reuse** ⇨ compose small parts instead of inheriting
- **Flexibility** ⇨ swap the inner type
- **Loose coupling** ⇨ depend on traits, not parents





Exercise - Composition





Rust has no class inheritance - reuse a `Lamp` through **composition**.

1. A `Room` **has-a** `Lamp` (a field) plus a `name: String`.
2. `Room::new(name)` builds a room whose lamp is **off**.
3. **Delegate**: `Room::turn_on(&mut self)` forwards to the lamp's `turn_on()`.
4. In `main`, build a room, turn it on, print whether its lamp is on.



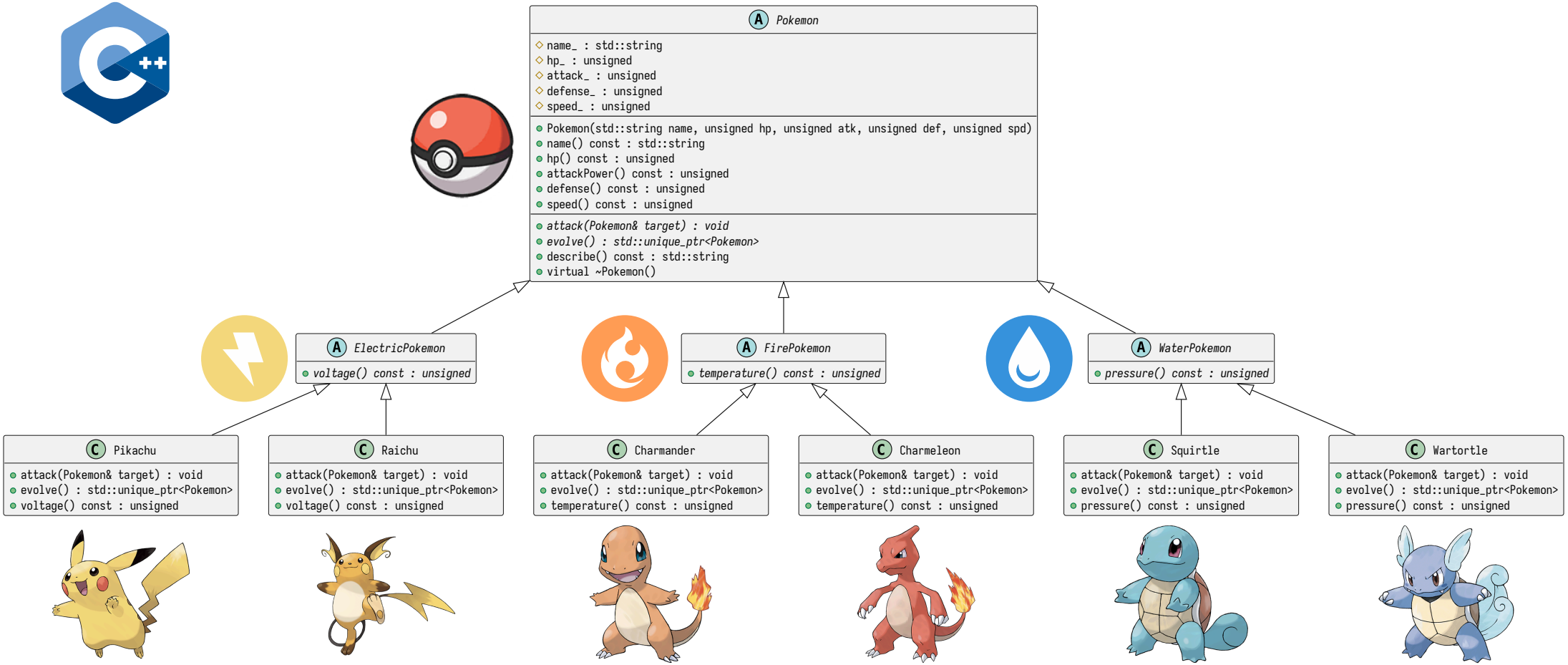
Has-a (a field), not **is-a** (a subclass): the room **delegates** to the lamp it owns.



Pillar 4 - Polymorphism

One name, many shapes

Polymorphism: One Name, Many Types



Polymorphism: One Name, Many Types



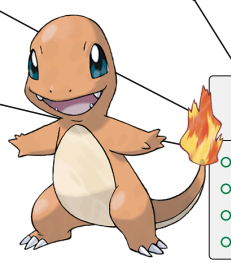


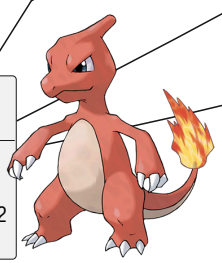


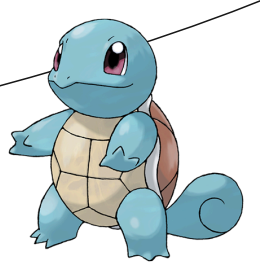










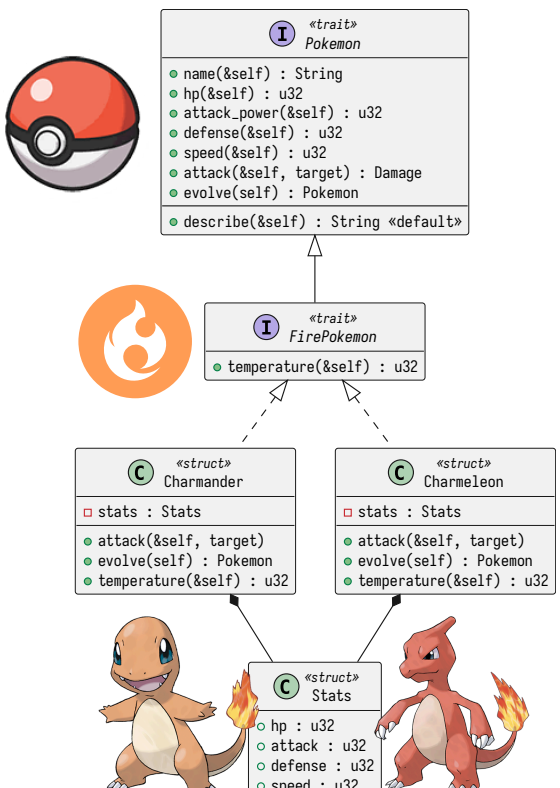




Polymorphism: One Name, Many Types



The **same** call works for **any** type that implements the trait. Rust resolves it two ways - at **compile time** or at **run time**.



- **One call, many types** - write `p.attack(t)` once
  a `Pikachu` thunderbolts, a `Charmander` embers
- Rust resolves that one call in one of **two ways**:
  - **Static** (compile time) ⇨ **generics + trait bounds** `T: Pokemon`
  - **Dynamic** (run time) ⇨ **trait objects** `dyn Pokemon`
- The call **looks identical** - only **how the right method is found** differs
- In any case: **substitutability**, one function serves every species

Static Dispatch: Generics & Trait Bounds



A **generic** with a **trait bound** `T: Pokemon` is compiled into one specialised copy **per concrete type**, resolved at **compile time**.

```rust
trait Pokemon { fn hp(&self) -> u32; }

struct Pikachu { hp: u32, }
impl Pokemon for Pikachu {
  fn hp(&self) -> u32 {
    self.hp
} }

// means T can be any type that implements Pokemon
fn total_hp<T: Pokemon>(team: &[T]) -> u32 {
  team.iter().map(|p| p.hp()).sum()
}

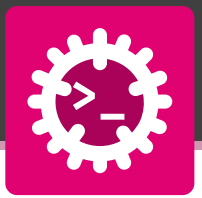
fn main() {
  // a slice holds ONE species, so T is fixed to Pikachu
  let squad = [
    Pikachu { hp: 35 }, Pikachu { hp: 40 },
  ];
  let total = total_hp(&squad); // → total_hp::<Pikachu>
  println!("{total}");          // 75
}
```

- **Trait bound** `T: Pokemon` constrains the generic
- **Monomorphization** - one copy generated per species used:

```rust
// conceptually generated:
fn total_hp_pikachu(&[Pikachu]) -> u32 {...}
fn total_hp_charmander(&[Charmander]) {...}
```

- Calls **inlined** ⇨ fastest, no indirection
- Cost: **larger** binary; a slice holds **one** species only
- Similar to C++ **templates**

Dynamic Dispatch: Trait Objects



A **trait object** `dyn Pokemon` hides the concrete species behind a pointer; the right method is found at **run time** via a **vtable**.

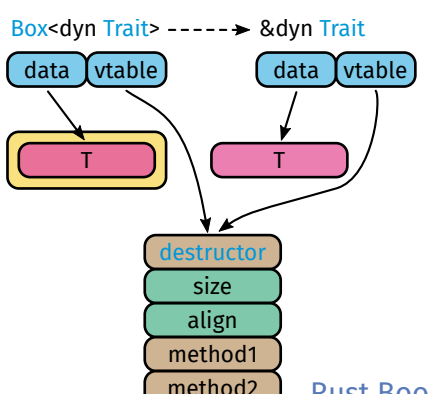
```rust
trait Pokemon { fn hp(&self) -> u32; }
struct Pikachu { hp: u32 }
impl Pokemon for Pikachu { fn hp(&self) -> u32 { self.hp } }
struct Charmander { hp: u32 }
impl Pokemon for Charmander { fn hp(&self) -> u32 { self.hp } }

// ONE function, no generics - works on a trait object
fn total_hp(team: &[Box<dyn Pokemon>]) -> u32 {
    team.iter().map(|p| p.hp()).sum()
}

fn main() {
    // a real TEAM mixes species, so we need `dyn`
    let team: Vec<Box<dyn Pokemon>> = vec![
        Box::new(Pikachu { hp: 35 }),
        Box::new(Charmander { hp: 39 }),
    ];

    let total = total_hp(&team); // each hp() found via vtable
    println!("{total}");         // 74
}
}
```

- **Trait object** `dyn Pokemon`, behind `Box<dyn ...>` or `&dyn ...`
- `&dyn Pokemon` is a **fat pointer**: (**data ptr**, **vtable ptr**)
- **vtable** ⇨ run-time table of method addresses
- **Heterogeneous**: one `Vec` holds `Pikachu` **and** `Charmander`
- Cost: a pointer jump to the right method
- Similar to **virtual functions** in C++ / Java



Static vs Dynamic Dispatch



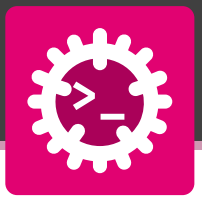
	Static - generics	**Dynamic** - trait objects
Syntax	`fn f<T: Pokemon>(t: &[T])`	`fn f(t: &[Box<dyn Pokemon>])`
Resolved	compile time (monomorphized)	run time (vtable)
Speed	fastest, inlinable	one pointer indirection
Binary size	larger (a copy per species)	smaller (one shared copy)
Collection	one species per slice	mixed species in one `Vec`
Classic OOP	templates	virtual methods



Rule of thumb: reach for **generics** by default; switch to **trait objects** when one collection must hold **different species** - a real team.

Exercise - Polymorphism





A home holds **different** device types in **one** list - that needs **dynamic dispatch**. Reusing the `Device` trait, **read** the code and give the output.

```
fn main() {
  let home: Vec<Box<dyn Device>> = vec![
    Box::new(Lamp { on: true }),
    Box::new(Thermostat { celsius: 22 }),
    Box::new(Lamp { on: false }),
  ];
  for d in &home {
    println!("{}", d.status()); // ?
  }
}
```



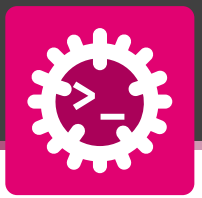
One species per list ⇨ **generics** (static). A **mix** of species ⇨ `Box<dyn Device>` (dynamic, via vtable).



OOP in Rust - Recap

Same goals, different tools

OOP in Rust - Recap



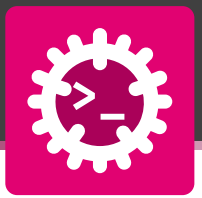
OOP concept	Rust mechanism
Class	`struct` / `enum` + `impl`
Object / instance	a value
Attribute / field	`struct` field
Method	`fn(&self)` in an `impl`
Constructor	associated `fn new() -> Self`
Encapsulation	modules + `pub` (private by default)

OOP concept	Rust mechanism
Interface / abstraction	`trait`
Abstract method	required trait method
Virtual method / override	default trait method
Inheritance	*none* - traits + composition
Subtype polymorphism	trait objects `dyn Trait`
Generics / templates	generics + trait bounds `T: Trait`



The four pillars survive; only **inheritance** is reshaped into **traits** (behaviour) and **composition** (data)

OOP in Rust - Recap



A trait is an interface: the set of methods a type promises to provide.

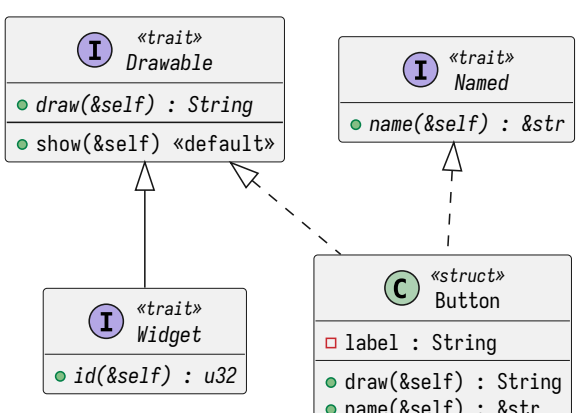
```rust
trait Drawable {
  fn draw(&self) -> String;     // required (abstract)
  fn show(&self) {              // default method
    println!("[{}]", self.draw());
  }
}
trait Named { fn name(&self) -> &str; }

struct Button { label: String }

// one type implements several interfaces
impl Drawable for Button {
  fn draw(&self) -> String { format!("Button({})", self.label) }
}
impl Named for Button {
  fn name(&self) -> &str { &self.label }
}

// supertrait: a Widget must also be Drawable
trait Widget: Drawable {
  fn id(&self) -> u32;
}
```

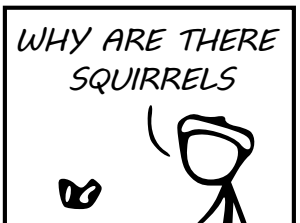
- **Abstract method** - required, no body
- **Default method** - has a body
- **Implementation** - `impl Trait for Type`
- **Multiple interfaces** - many traits per type
- **Supertrait** - `trait A: B`
- **Dependency inversion** - depend on trait bounds, not concrete types



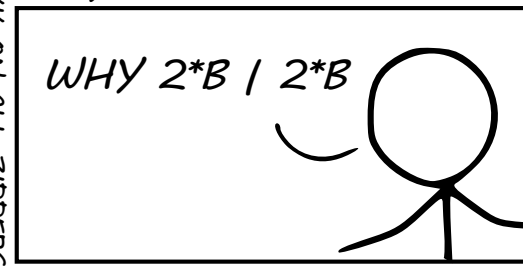
QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME

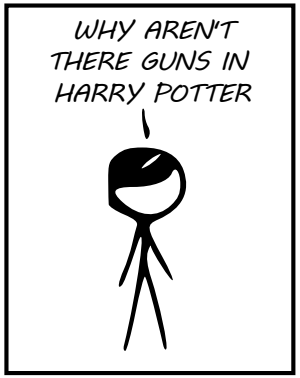
WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST·EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES
WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH



WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS
WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR

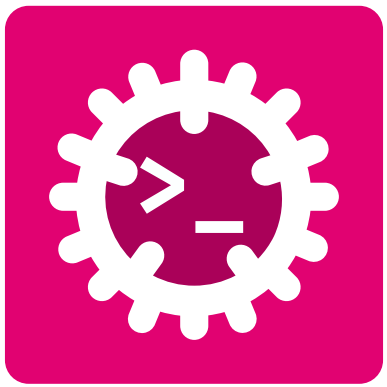


WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY ARE THERE ANTS IN MY LAPTOP
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILEGRIM
WHY ARE THERE SO MANY CROWS IN ROCHES
WHY IS TO BE OR NOT TO BE FU
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE

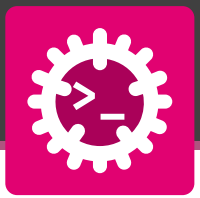


WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHAT IS https://xkcd.com/1256/
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE

WHY DO IGUANAS DIE
WHY AREN'T THERE DINOSAUR GHOSTS
WHY IS THERE LAVA
WHY ARE THERE FEMA
WHY IS THERE HE
WHY IS YKK ON ALL ZIPPERS

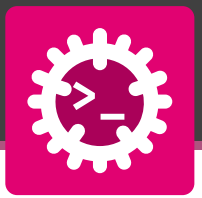


Glossary & Bibliography



Programming

***OOP* – Object-Oriented Programming**: Programming paradigm based on abstraction, encapsulation, modularity and hierarchy. 1, 1, 3, 7, 8, 8, 38, 40, 41, 41, 41, 42



Bibliography

[1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”

[2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.

[3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.

[4] A. Shvets, “Refactoring Guru.” 2026.

[5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[6] “SVG Repo - Free SVG Vectors and Icons.”